Is the Map Right?

An execution-grounded audit of the code graph and embeddings handed to local SWE agents

Gemma 4 Developer Agent Competition, Paper Track

October 2026

Abstract

Coding agents are increasingly given a pre-built map of the repository: a call graph and an embedding index. Work on such tools reports downstream accuracy but never checks the map against what the code does. We audit the graph and embeddings distributed to every entrant of the Gemma 4 Developer Agent Competition, 127 graphs covering 129 tasks from fastapi, rich, requests and httpx, using each task's own tests as the oracle. We parse every source file, trace every task's tests before and after the reference fix, and read the caller's source for every disputed edge. First, the graph contains no async functions: 0 of 45,492 async definitions are nodes, which removes 81 of 711 gold-patch symbols and touches 31% of fastapi tasks. Second, the harness's name resolver turns these omissions into wrong answers: a bare-name query for an absent async function returns a different node 65% of the time, usually a test. Third, the `calls` relation captures 37% of observed calls between existing nodes, and only 21% of static edges between executed functions were ever exercised; the caller's source shows that most unexercised edges are containment and inheritance links stored under the `calls` label, and most misses are unresolved method dispatch. Fourth, the embeddings are not reproducible from any EmbeddingGemma checkpoint, and the shipped symbol-to-symbol search finds the gold file half as often as BM25 over the source. The graph does connect failing tests to the code that must change: from where a failing test enters the library, every executed gold function is reachable in 69 of 72 tasks. We release the toolkit, a corrected typed graph and per-task annotations.

1 Introduction

The competition asks whether a 4-bit, 31-billion-parameter model on consumer hardware can fix real Python bugs [16]. Alongside the tasks, the organizers ship two structural aids: a repository graph exposed through `get_code_neighbors` and `get_code_subgraph`, and per-node embeddings behind `search_similar_code`. Every entrant's agent can call them, and the hidden test set uses the same generator, so properties of these artefacts apply to every submission.

Recent work says structure helps agents when it is delivered well. In-band graph neighbours added 11.8 points of file accuracy over a strong baseline [7]; structural comments added 2.2 points of function-level accuracy and halved run-to-run variance [6]; language-server-resolved relations were worth 7 points for a small Gemma model [8]. Every one of these papers builds its own graph and reports what the agent achieves with it. None measures whether the graph is correct, and agents adopt corrupted tool returns more than a third of the time [17], so an error in the map may cost more than its absence.

We therefore ask a prior question: *is the map right?* Using each task's tests as ground truth, we measure what the graph contains (§5.1), what an agent's queries return (§5.2), how its edges compare with execution and what they encode (§5.3, §5.4), whether it connects failing tests to the fix (§5.5), and what the embedding index retrieves (§5.6). We release the toolkit, a typed graph and per-task profiles (§6).

2 Related work

Static call graphs for Python. PyCG reports about 99% precision and 70% recall on its benchmarks by resolving assignments between identifiers [1]. Later work raises recall with LLM filtering [2], and a comparison of Code2flow, Pyan and WALA found none sound and many spurious edges [3]. These studies audit *tools*; we audit an *artefact handed to agents*, whose generator is undocumented.

Execution as the oracle. Measuring static recall against calls recorded while tests run is standard practice: Sui et al. report a median recall of 0.88 over 31 Java programs [4], and Helm et al. extend the baseline with fuzzing [5]. We follow it, using each task's own fail-to-pass tests.

Structure for coding agents. Graph tools offered separately tend to go unused, while the same structure placed in-band helps [6, 7, 9]. Dynamic traces lifted SWE-bench Verified resolution from 65.4% to 74.2%, but only when summarized [10]. Typed, resolved relations matter more than syntactic edges [8, 11], and localization gets harder with the hop distance from issue-mentioned code to the fix [12]. None of these works measures the error rate of the graph given to the agent.

Embeddings and retrieval for agents. How code is represented to a retriever changes what it finds by up to 40% Hit@5 [13], and compact free-text semantic search lets small localizers beat grep [14]. EmbeddingGemma 2 documents a native 256-dimensional truncation [15], the dimension of the competition's vectors. No study has measured what a symbol-to-symbol search loses against free text over the same repository.

3 The artefacts under audit

Each task carries a base commit, an issue text, a reference patch and a test patch. The graph for a base commit is a NetworkX node-link file whose nodes carry an id (`module.Class.method`), a name and the full source text. Across the 127 graphs there is one edge type, `calls`, with 452,588 edges. Each node has one 256-dimensional float32 vector. The harness resolves a symbol query in four tiers, exact, dotted suffix, case-insensitive and substring, and breaks ties by fewest dots, then shortest id. `search_similar_code` resolves its query to a node this way and ranks other nodes by cosine; it cannot take free text. The harness runs at a 32,768-token context.

4 Method

All steps are CPU-only and run under Python 3.13.5, the harness's interpreter version, on two 8- and 16-core virtual machines.

Static census. For every snapshot we parse every `.py` file with `ast`, enumerate sync functions, async functions, classes and nested definitions, map each to the graph's id scheme, and record presence. For each definition we issue the queries an agent would type, the bare name and `Class.method`, through a line-for-line copy of the harness resolver, and classify the outcome as *self*, *no match* or *wrong node*. Gold symbols are the innermost definitions enclosing each changed line of the reference patch.

Id reconciliation. The graph's id scheme is undocumented. We classify every provided node against the parsed source and every parsed definition against the graph. This revealed that nested definitions appear under a flattened id: enclosing functions and the `<locals>` marker are dropped, classes are kept. All other steps use this scheme.

Dynamic oracle. For each task we extract the snapshot into a fresh virtual environment, install the package and its test requirements (best effort, network allowed), apply the test patch, and run the target test files under a `sys.setprofile` and `threading.setprofile` tracer; then we apply the reference patch and run again. The tracer records every repository function executed per test and every caller-to-callee pair, directly or through library frames, counting coroutine and generator frames once. A task is *usable* when both runs are clean and at least one expected test fails before the fix and passes after: 91 of 129 tasks (fastapi 45, rich 39, requests 7). Edges are attributed from the pre-fix trace, since the graph describes the base commit. *Representable recall* is the share of observed calls between two existing nodes that the graph holds; *agent-facing recall* also counts calls to omitted definitions; *observed precision* is the share of static edges whose endpoints both executed that were observed as calls. Precision outside executed code is unknown. Intervals are 95% task-cluster bootstrap for pooled ratios and Wilson intervals for task-level rates.

Reachability. The failing tests are usually new in the test patch, so they are not nodes of the base graph. We seed the search instead from where each failing test enters the library: executed non-test nodes none of whose caller chains reaches another non-test node before test code, walking through frames that are not nodes. A directed breadth-first search on `calls` then measures the hops to the gold functions those tests execute.

Edge semantics. For every static edge never observed at runtime, every observed call the graph misses, and every captured call, we read the caller's own source and record how the callee appears: a bare call, a `self.` call, a call on another object, an instantiation, a reference without a call, a mention only in a string or

comment, implicit dunder dispatch, or no mention at all. We also classify each edge by the structural relation between its endpoints in the typed graph.

Embeddings and retrieval. We re-embed 200 nodes from each of 10 graphs with EmbeddingGemma 300m, EmbeddingGemma 2 and a MiniLM control, over several input variants and prompts at 256 dimensions; mean cosine of 0.99 or more counts as identified. We measure whether cosine separates call-adjacent, containment, gold co-change, same-module and name-overlap pairs from random pairs. Finally we compare five retrieval modes per task: resolved issue identifiers only, the harness mode (identifiers plus cosine), harness plus one-hop neighbours, BM25 over node text, and BM25 over the real source files.

5 Results

5.1 The graph has no async functions

Figure 1 gives presence by kind. Restricted to non-test source files, sync functions are 100% present and async functions 0 of 28,113. The missing classes and sync functions are id-scheme effects that the reconciliation accounts for in full: fastapi lists 1,683 inner pydantic `Config` classes without their model class, and requests lists a subset of test methods without their test class. Every missing nested definition in fastapi and rich is itself async; in requests a further 340 are methods of helper classes defined inside test functions.

The omission falls on the code that needs fixing. Of 711 gold-patch symbols, 81 are async and none is a node; one appears present only because the nested async `FastAPI.setup.<locals>.openapi` flattens to the id of a different, sync method, `FastAPI.openapi`. Overall 470 of 711 gold symbols (66.1%, CI 62.6 to 69.5) are nodes. 22 tasks have an async gold symbol, 21 of them in fastapi, which is 31% of fastapi tasks. In 3 usable tasks a gold function that the failing test executes has no node at all. fastapi is an async web framework, and for its request-handling path the graph is silent.

5.2 The resolver turns omissions into wrong answers

An agent that asks for an absent function does not get "not found" (Figure 2). For a bare-name query on an absent async function, the resolver returns a different node in 65.4% of 56,487 cases, a test node in 26,851 of them, and nothing in the remaining 34.6%. The problem is not limited to absent code: for sync functions that *are* nodes, the bare name resolves to a different, same-named node 41.2% of the time (46.6% in fastapi, 62.0% in httpx), and 47.9% for classes, because the tie-break prefers short ids and fastapi's tutorial and test modules reuse names. For 21 of 129 tasks, the most specific query for some gold symbol lands on the wrong node.

5.3 The `calls` edges against execution

Table 1 and Figure 3 give the comparison. Of the observed calls the graph misses, 79% run between two existing nodes, 11% touch an omitted sync definition, 6% a nested function and 4% an absent async function. Recall is 65.6% in fastapi, 30.5% in rich and 34.7% in requests; precision is lowest in fastapi (16.5%) and highest in requests (34.3%). The tests executed 16.7% of non-test nodes, so all figures describe the code paths the tasks exercise.

This contradicts our hypothesis that the graph would be precise but incomplete, like conservative static analysers [1]: it is incomplete *and* most of its edges between executed functions never ran.

5.4 What the `calls` relation encodes

Reading the caller's source explains both numbers (Figure 4). Captured edges have an explicit call site 88.6% of the time, mostly bare calls (55.3%) and instantiations (19.0%): what a name-matching walker finds. Missed calls are method dispatch that the walker does not resolve: calls on other objects (23.4%), `self.` calls (15.9%), references passed as callbacks (28.3%) and implicit dunder calls (17.3%). The 35,017 unexercised edges mostly have no textual basis in the caller: in 56.1% the callee is not mentioned at all and in 26.7% only in a string or comment.

Figure 5 shows what they are instead. Of all unexercised non-test edges, 22.3% point from a method to its own class, 13.8% from a class to a subclass and 13.3% from a class to a base class; among missed calls, 45.5% run between two methods of the same class. We infer that the generator stores containment and inheritance, in both directions, under the label `calls`, which also explains why 8 to 21% of `calls` edges per graph point from a symbol to its own member. An agent following "callers" and "callees" is in part walking a class hierarchy.

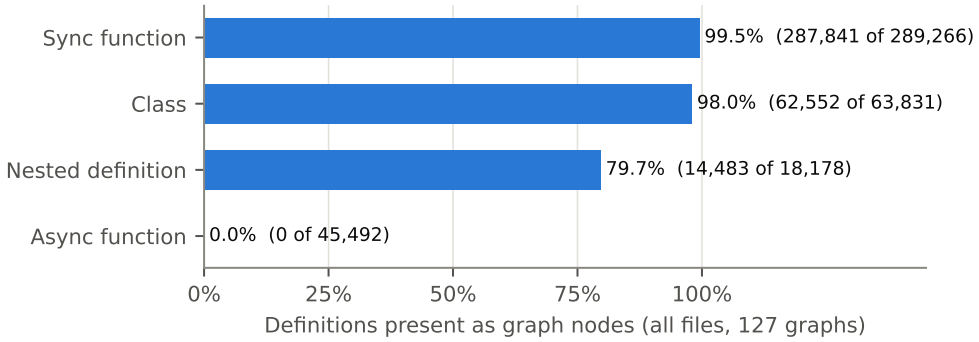


Figure 1: Share of definitions in every snapshot that exist as graph nodes, by kind. Async functions are absent from all 127 graphs.

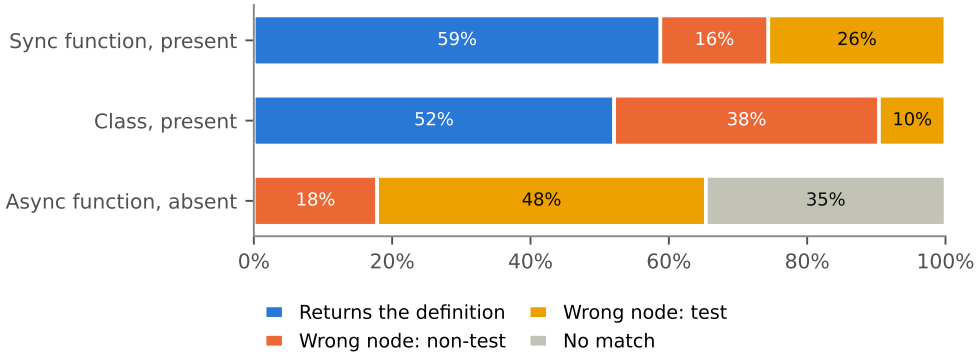


Figure 2: What the harness resolver returns for a bare-name query: 287,841 present sync functions, 62,552 present classes, 56,487 absent async functions.

5.5 The graph connects failing tests to the fix

Despite the mislabelled and missing edges, the graph does link the code a failing test touches to the code that must change. An entry set exists for 86 of 91 usable tasks, and 72 tasks have a gold function that the failing tests execute and that is a node. In 69 of these 72 (95.8%, CI 88.5 to 98.6) every such gold function is reachable from the entry set along directed `calls` edges; none is reachable only against edge direction. Figure 6 gives the distances: the gold function is itself an entry point in 32 tasks and within three hops in 63.

We had hypothesized that gold functions would often be reachable at runtime but not in the static graph. On this data the hypothesis is rejected. The obstacles for an agent are omission (async code is absent and cannot be reached), mislabelling (much of a node's frontier is class structure) and distance on a hub-heavy graph, not missing connectivity.

5.6 Embeddings and retrieval

The encoder is not public EmbeddingGemma. No candidate approaches identification: the best mean cosine to the provided vectors is 0.073 for EmbeddingGemma 300m (name variant, document prompt), 0.015 for EmbeddingGemma 2 and 0.036 for the MiniLM control, with pairwise rank correlation at most 0.16. The provided vectors are not unit-normalised and are strongly anisotropic: mean cosine to the mean vector is 0.87 to 0.95, the first principal component explains 44% to 75% of the variance, and the top-1 neighbour cosine averages 0.999.

What similarity tracks. Cosine separates containment and call-adjacent pairs from random pairs well (AUROC 0.90), gold co-change pairs less (0.81), same-module and name-overlap pairs weakly (0.68; Figure 7a). But a node's ten nearest neighbours are 45% to 67% test nodes and only 2% to 6% call neighbours (Figure 7b), so the tool mostly returns tests.

Retrieval. Figure 8 compares the five modes over all 129 tasks. The cosine step changes three tasks and one-hop expansion two more. Free-text BM25 over the source finds the gold file in 47 tasks where the harness mode does not, and the reverse holds for 5 (exact McNemar $p < 0.0001$). The harness mode returns nothing for 54 of 129 tasks, because no identifier in the issue resolves to a node.

Usable tasks (91)	Value	95% CI
Representable recall	37.1% (8,032 of 21,653)	34.8 to 40.2
Agent-facing recall	31.8% (8,032 of 25,267)	29.4 to 34.8
Recall on gold-incident calls	56.3% (604 of 1,073)	39.3 to 68.4
Observed precision	21.4% (8,013 of 37,433)	20.0 to 22.8

Table 1: The provided `calls` edges against calls observed while the tasks' tests run.

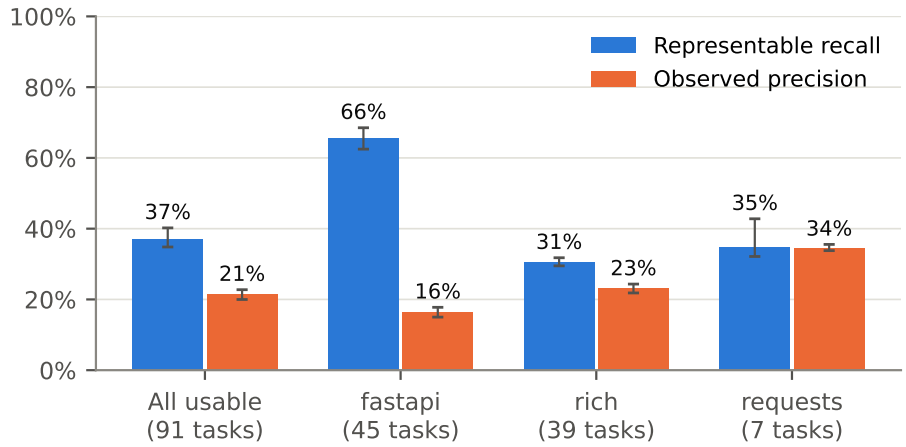


Figure 3: Representable recall and observed precision, pooled and per repository, with 95% task-cluster bootstrap intervals.

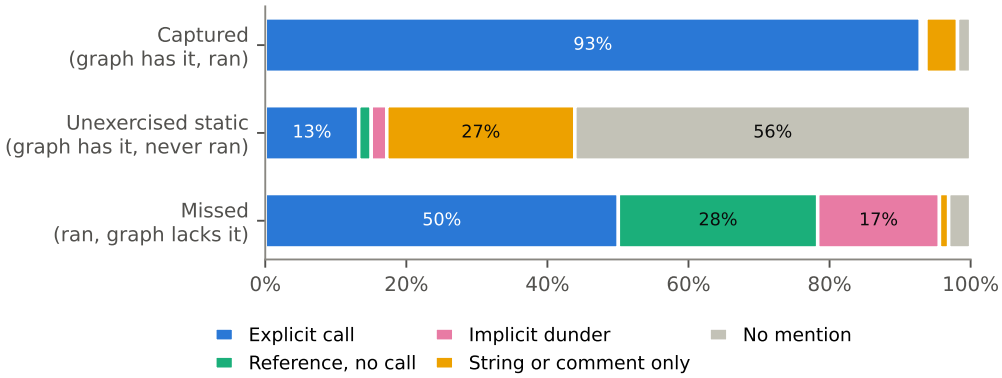


Figure 4: How the callee appears in the caller's own source, for non-test edges: 7,640 captured, 35,017 unexercised static, 12,231 missed.

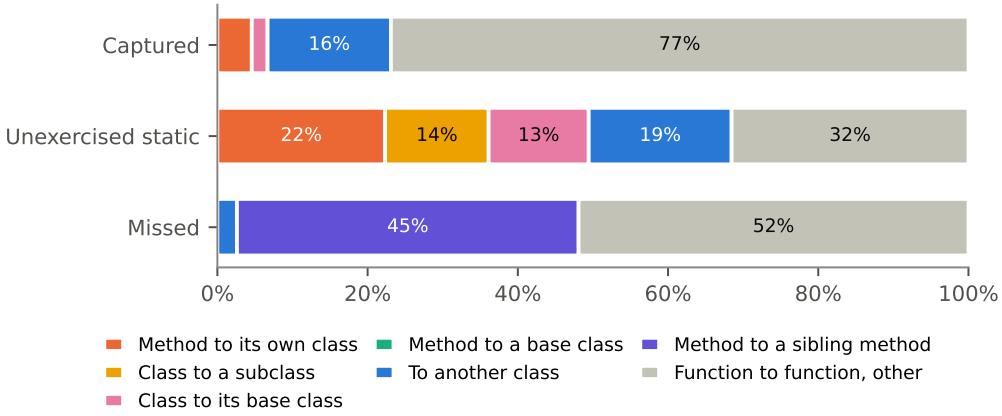


Figure 5: Structural relation between the endpoints of every non-test edge in the three populations of Figure 4.

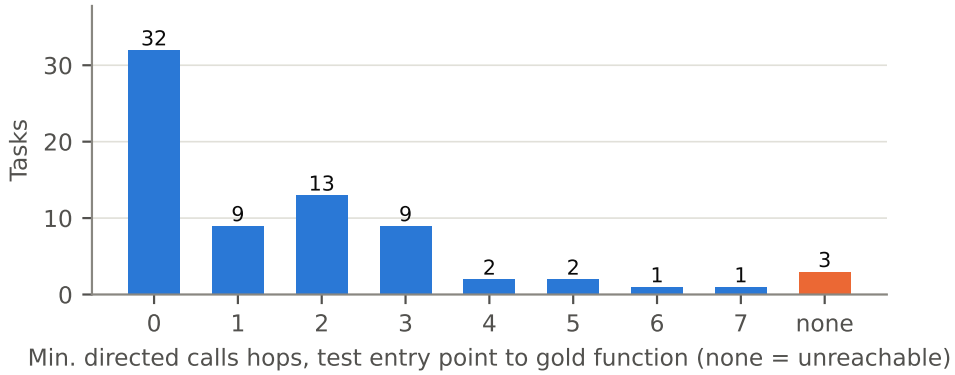


Figure 6: Minimum directed `calls` hops from where the failing tests enter the library to a gold function they execute (72 usable tasks with an executed gold node).

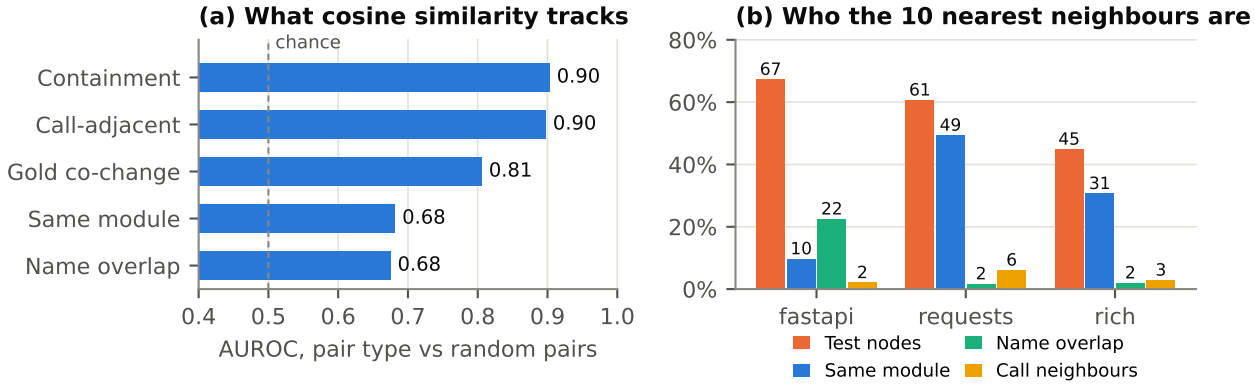


Figure 7: (a) Mean AUROC over 127 graphs for separating each pair type from random pairs by cosine. (b) Composition of each node's 10 nearest neighbours.

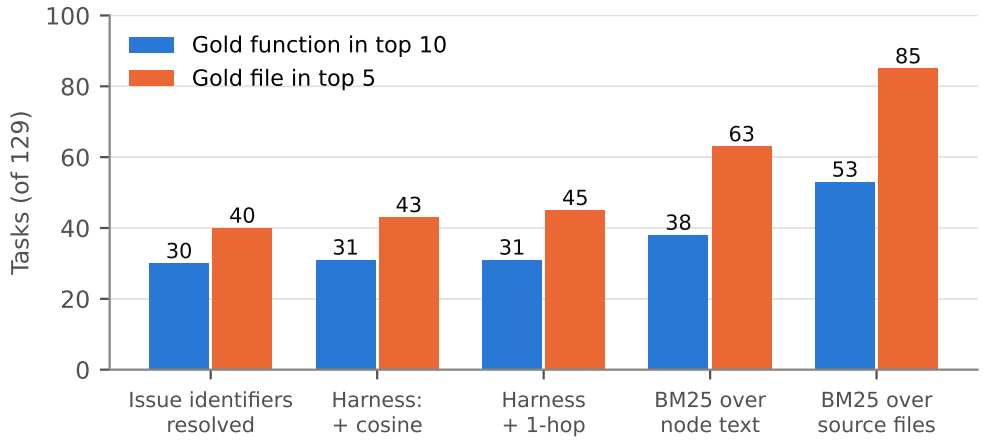


Figure 8: Tasks (of 129) where each mode ranks a gold function in its top 10 or a gold file in its top 5.

6 Released resources

- **Audit toolkit**: every step above, with a synthetic-fixture test suite (15 tests).
- **Typed graph v2** per base commit, in the harness's format and ids: functions *including async*, nested definitions, and typed edges `contains`, `imports`, `inherits`, `calls_static`, `calls_dynamic` (with test provenance) and `tests`.
- **Per-task profile**: gold coverage, async exposure, resolver hazards, entry distance, reachability, recall and precision.

7 Implications and limitations

For agent builders. Treat `get_code_neighbors` output as a mix of call and class-structure edges, check the id a resolved name returns, and use text search for async code; a sandbox Skill script can serve the typed graph legally.

For benchmark designers. Document the graph's semantics, split containment, inheritance and calls into typed edges, return "not found" instead of a substring match, and include async definitions.

Limitations. Precision is a lower bound measured only inside executed code. Our runs used per-task environments with network access rather than the official cached dependencies; 9 collection errors, 2 import errors and 4 requests tasks without collectable tests are our setup, not the tasks. Recall covers the paths the tests exercise. The generator's rules are inferred from its outputs, and the 80-edge hand-review sample is not yet labelled. The hidden test set uses the same generator, but we have not measured it. Results come from four repositories, two of which dominate the task count.

Reproducibility

Every number comes from result files under `planA_results/vm-a2/results/` (`a1_v2`, `a7`, `a2_v3`, `a8`, `a3`, `a5_v3`, `a6_v3`) and `vm-main/results/a4`. `scripts/ops/` regenerates them from the competition archive; `papers/make_figures.py` draws every figure.

References

[1] V. Salis et al. PyCG: Practical call graph generation in Python. ICSE, 2021. https://consensus.app/papers/details/b2f145e1fe035745a8b387e3f4b74740/

[2] Y. Xiang et al. InferCG: Enhancing Python call graph generation via static analysis and LLMs. TOSEM, 2026. https://consensus.app/papers/details/f9176ef478f6583eaef6c696919949a8/

[3] S. Kummita et al. Qualitative and quantitative analysis of callgraph algorithms for Python, 2021. https://consensus.app/papers/details/2565e0e7f78a5d05b74121697239f917/

[4] L. Sui et al. On the recall of static call graph construction in practice. ICSE, 2020. https://consensus.app/papers/details/93f20309e7855a5898b1f0f293a01442/

[5] D. Helm et al. Total recall? How good are static call graphs really? ISSTA, 2024. https://consensus.app/papers/details/c62e543287615a7abeb7430f76391389/

[6] Lin et al. How much static structure do code agents need? (CodeAnchor). ISSTA, 2026. https://www.alphaxiv.org/abs/2606.26979

[7] Hu et al. LARGER: Lexically anchored repository graph exploration and retrieval, 2026. https://www.alphaxiv.org/abs/2605.16352

[8] Wei et al. SemNav: Semantic navigation for repository-level issue localization, 2026. https://www.alphaxiv.org/abs/2609.31176

[9] Paipuru. The navigation paradox in large-context agentic coding (CodeCompass), 2026. https://www.alphaxiv.org/abs/2602.20048

[10] Liu et al. DAIRA: Dynamic analysis-enhanced issue resolution agent, 2026. https://www.alphaxiv.org/abs/2603.22048

[11] Wang et al. GREPO: A benchmark for GNNs on repository-level bug localization, 2026. https://www.alphaxiv.org/abs/2602.13921

[12] Liu et al. GraphLocator: Graph-guided causal reasoning for issue localization, 2025. https://www.alphaxiv.org/abs/2512.22469

[13] Caumartin et al. Retrieval-oriented code representations in agentic bug localization, 2026. https://www.alphaxiv.org/abs/2607.11046

[14] Lasbordes et al. Small agents with semantic search: Efficient multilingual code localization, 2026. https://www.alphaxiv.org/abs/2610.05099

[15] Google DeepMind. EmbeddingGemma 2 model card, 2026. https://huggingface.co/google/embeddinggemma-2

[16] Gemma Team. Gemma 4 technical report, 2026. https://www.alphaxiv.org/abs/2607.02770

[17] Yang et al. Agents' overreliance on unreliable tools, 2026. https://www.alphaxiv.org/abs/2609.05587